

CS 6220: Homework 2

Linear Regression and Regression Trees

Name: Nikhil Singh Shekhawat

Q1. Source-code repository

GitHub: <https://github.com/sh-nikhil/CS6220-Linear-Regression-and-Regression-Trees.git>

The repository contains:

- `HW2.ipynb`: all code, already executed so every output is visible. It is organized in markdown-headed parts: setup, data generator, linear regression, regression tree, linear-regression experiments, tree experiments, and the results dump.
- `README.md`: build and run instructions.
- `requirements.txt`: dependencies.
- `figures/`: all plots.
- `results/results.json`: all numbers quoted below.

Data setup used throughout. `generate_noisy_dataset` samples $x \sim U[0,1]$, builds the columns $(x, f_1(x), \dots, f_d(x))$, and sets $y = f(x) + U[-\epsilon, \epsilon]$.

- A fixed seed (42) is used, and x is drawn before the noise. As a result the schemas (x,y) and (x,x^2,x^3,y) contain *identical* x and y values.
- Every dataset is split once with a fixed permutation (seed 7) into 800 training and 200 test records. Every linear model and every tree for a given (f, ϵ) therefore uses exactly the same training and test records.
- Because the seed is shared, the linear and cubic datasets also share the same x values and noise draws. This is why some RMSEs below agree to all digits.

Q2. Pseudo-code: linear regression

```
function GENERATE_NOISY_DATASET(feature_funcs, f, [a,b], n, ε, seed):
    x ← n samples from Uniform(a, b)
    X ← columns [x, f_1(x), ..., f_d(x)]          # d = |feature_funcs|, may be 0
    y ← f(x) + n samples from Uniform(-ε, ε)
    return X, y

function TRAIN_TEST_SPLIT(X, y, seed):          # fixed partition
    π ← random permutation of 0..n-1 (seeded)
    return X[π[0:0.8n]], X[π[0.8n:]], y[π[0:0.8n]], y[π[0.8n:]]

function FIT(X_train, y_train):
    X ← [1 | X_train]                          # step 1: prepend column of 1s; Y ← y_train
    β ← (XT X)-1 XT Y                        # step 2: normal equation (numpy)
    return β                                    # step 3: weight vector (β0 = intercept)

function PREDICT(β, X_test):
    X ← [1 | X_test]
    return X β                                # ŷi = β · xi for every record

function EVALUATE_MODEL(ŷ, y):
    return sqrt( Σi (yi - ŷi)2 / n )      # RMSE
```

Q3. Linear regression on (x, y)

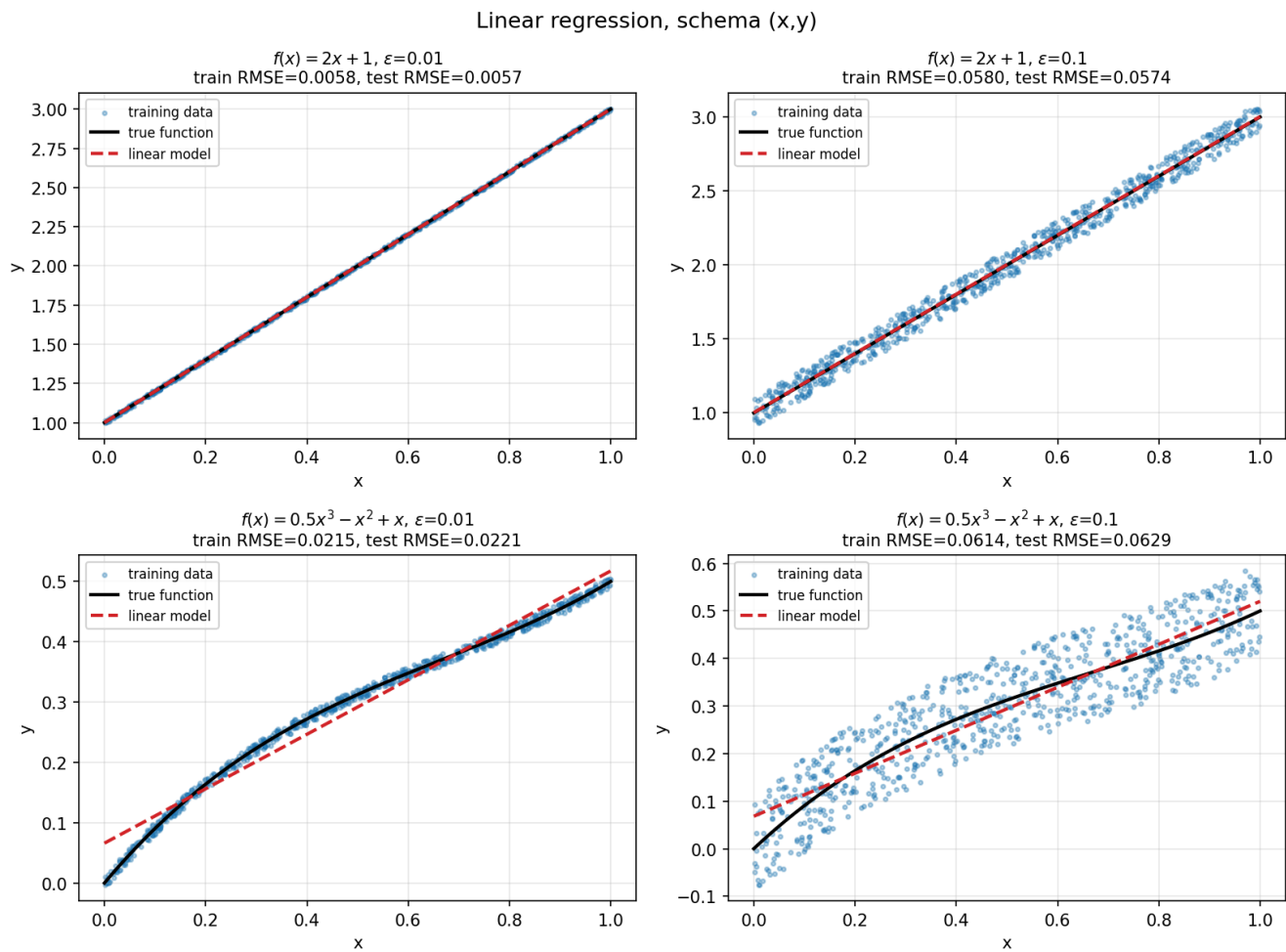


Figure 1. Schema (x, y): training data (800 points), true function (black), and learned linear model (red, dashed).

True f	ϵ	Learned model $\hat{f}(x)$	Train RMSE	Test RMSE
Linear $2x+1$	0.01	$1.0002 + 2.0001x$	0.00580	0.00574
Linear $2x+1$	0.1	$1.0024 + 2.0006x$	0.05796	0.05739
Cubic $0.5x^3 - x^2 + x$	0.01	$0.0661 + 0.4515x$	0.02149	0.02214
Cubic $0.5x^3 - x^2 + x$	0.1	$0.0682 + 0.4520x$	0.06142	0.06287

The straight line matches the linear truth almost perfectly. For the cubic truth it cannot follow the curvature: it underestimates near both ends of the range and overestimates in the middle. This bias shows up as a test RMSE of 0.0221 at $\epsilon = 0.01$, about four times the noise level.

Q4. Linear regression on (x, x^2, x^3, y)

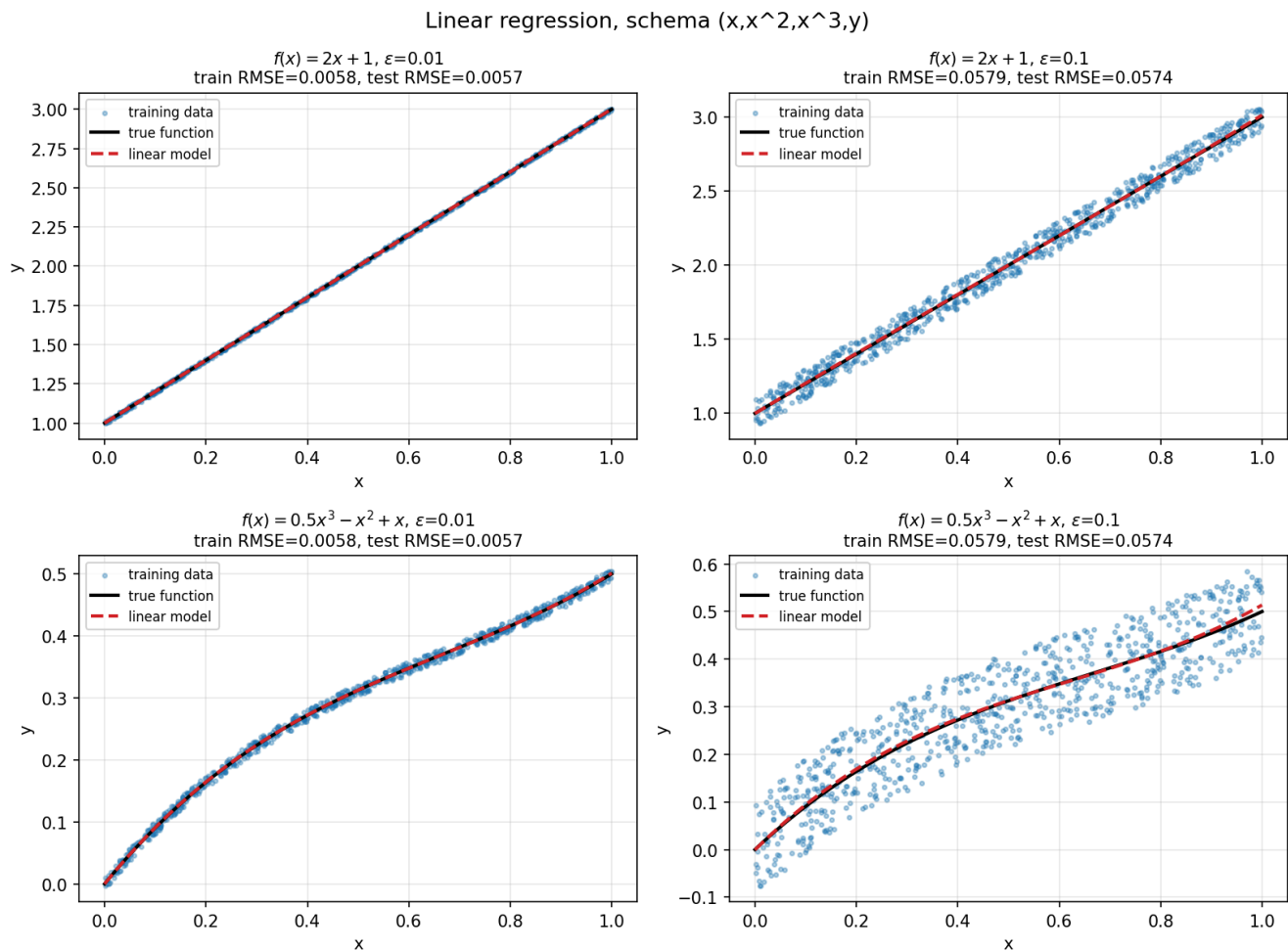


Figure 2. Schema (x, x^2, x^3, y) : the full learned function $\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$ is plotted.

True f	ϵ	Learned model $\hat{f}(x)$	Train RMSE	Test RMSE
Linear $2x+1$	0.01	$0.9999 + 2.0059x - 0.0179x^2 + 0.0135x^3$	0.00579	0.00574
Linear $2x+1$	0.1	$0.9994 + 2.0587x - 0.1793x^2 + 0.1350x^3$	0.05788	0.05744
Cubic $0.5x^3 - x^2 + x$	0.01	$-0.0001 + 1.0059x - 1.0179x^2 + 0.5135x^3$	0.00579	0.00574
Cubic $0.5x^3 - x^2 + x$	0.1	$-0.0006 + 1.0587x - 1.1793x^2 + 0.6350x^3$	0.05788	0.05744

Do the extra columns help? Yes for the cubic function, and not for the linear function.

- **Cubic:** the model class now contains the true function. Test RMSE falls from 0.0221 to 0.0057 ($\epsilon = 0.01$) and from 0.0629 to 0.0574 ($\epsilon = 0.1$), which is the noise floor in both cases (see Q6c).
- **Linear:** the extra columns bring no gain. The true function already lies in the simpler model class, so the model only spends its x^2 and x^3 coefficients fitting noise (e.g. $-0.18x^2 + 0.14x^3$ at $\epsilon = 0.1$). Test RMSE is unchanged or marginally worse ($0.05739 \rightarrow 0.05744$).

The deviations of the cubic-schema coefficients from the truth are identical for both true functions ($+0.0587, -0.1793, +0.1350$ at $\epsilon = 0.1$). The reason is that $f_{\text{cubic}} - f_{\text{linear}}$ lies in the span of the columns and the two datasets share the same noise draws, so least squares fits exactly the same noise in both cases.

Q5. Prediction error at $x = 10$

Schema	True f	ϵ	f(10)	Prediction	Error $\hat{y}-f(10)$	Rel. error
(x, y)	Linear $2x+1$	0.01	21	21.001	+0.001	0.0 %
(x, y)	Linear $2x+1$	0.1	21	21.008	+0.008	0.0 %
(x, y)	Cubic $0.5x^3-x^2+x$	0.01	410	4.581	-405.419	98.9 %
(x, y)	Cubic $0.5x^3-x^2+x$	0.1	410	4.588	-405.412	98.9 %
(x, x^2 , x^3 , y)	Linear $2x+1$	0.01	21	32.767	+11.767	56.0 %
(x, x^2 , x^3 , y)	Linear $2x+1$	0.1	21	138.667	+117.667	560.3 %
(x, x^2 , x^3 , y)	Cubic $0.5x^3-x^2+x$	0.01	410	421.767	+11.767	2.9 %
(x, x^2 , x^3 , y)	Cubic $0.5x^3-x^2+x$	0.1	410	527.667	+117.667	28.7 %

The prediction is only **good** for the **linear true function with the (x, y) schema**, with errors of 0.001 and 0.008. There the model is the correct form and its two coefficients are estimated almost exactly, so extrapolation stays accurate.

- **Cubic truth, (x, y) schema:** the straight line cannot represent the x^3 growth and predicts about 4.6 instead of 410.
- **(x, x^2 , x^3) schema, both truths:** the small coefficient errors from fitting noise on $[0, 1]$ are multiplied by $x^2 = 100$ and $x^3 = 1000$. The prediction is off by 11.8 ($\epsilon = 0.01$) or 117.7 ($\epsilon = 0.1$).

The $\epsilon = 0.01$ cubic model is still within 3 %, but in general extrapolating far outside the training range is unreliable, especially for higher-degree models.

Q6. Analysis across all linear-regression experiments

(a) For which true function does the model predict well?

For the **linear** function the model predicts well with either schema, with test RMSE at the noise floor. For the **cubic** function it predicts well only with the extra columns x^2, x^3 . With (x, y) alone it has clear bias: the test RMSE is 3.9× the noise floor at $\epsilon = 0.01$, and still 9 % above it at $\epsilon = 0.1$, where the noise hides much of the bias.

(b) Optimal model

Under squared loss the optimal predictor is the conditional mean $E[y | x]$. Here $y = f(x) + \text{noise}$, and the noise is $U[-\epsilon, \epsilon]$ with mean 0 and independent of x , so $E[y | x] = f(x)$. **The optimal model is the true function itself:**

- Linear data: $f^*(x) = 1 + 2x$.
- Cubic data: $f^*(x) = x - x^2 + 0.5x^3$.

It has zero bias and zero variance and leaves only the noise.

Learned models compared with the optimum:

- **Linear truth, (x, y) :** $1.0002 + 2.0001x$ ($\epsilon = 0.01$) and $1.0024 + 2.0006x$ ($\epsilon = 0.1$), essentially the optimal model.
- **Cubic truth, (x, x^2, x^3) :** $-0.0001 + 1.0059x - 1.0179x^2 + 0.5135x^3$ at $\epsilon = 0.01$ is very close. At $\epsilon = 0.1$ ($-0.0006 + 1.0587x - 1.1793x^2 + 0.6350x^3$) the coefficients are noticeably off, because x, x^2 and x^3 are highly correlated on $[0, 1]$. The resulting curve is still almost indistinguishable from f on $[0, 1]$ (Figure 2).
- **Cubic truth, (x, y) :** cannot reach the optimum, because no straight line equals the cubic.

(c) Irreducible error

The irreducible error is the error of the optimal model, i.e. the noise itself: $E[(y - f(x))^2] = \text{Var}(U[-\epsilon, \epsilon]) = (2\epsilon)^2/12 = \epsilon^2/3$. As an RMSE this is $\epsilon/\sqrt{3}$. We also measured it empirically as the RMSE of the true f on each test set.

True f	ϵ	Irreducible RMSE $\epsilon/\sqrt{3}$	RMSE of true f on test set	Test RMSE, model on (x, y)	Test RMSE, model on (x, x^2, x^3, y)
Linear $2x+1$	0.01	0.00577	0.00572	0.00574	0.00574
Linear $2x+1$	0.1	0.05774	0.05723	0.05739	0.05744
Cubic $0.5x^3 - x^2 + x$	0.01	0.00577	0.00572	0.02214	0.00574
Cubic $0.5x^3 - x^2 + x$	0.1	0.05774	0.05723	0.06287	0.05744

Every model whose hypothesis class contains the true function reaches the irreducible error on the test data, within 0.5 %:

- the linear truth with either schema;
- the cubic truth with (x, x^2, x^3) .

The cubic truth with (x, y) stays above the floor, with 0.0221 vs 0.0058 and 0.0629 vs 0.0577, because of its bias. No model can go below the floor on average. The small differences (e.g. 0.00574 vs 0.00577) are

sampling fluctuation of the 200 test records.

Q7. Adding higher-order monomials (cubic, $\varepsilon = 0.1$, $n = 1000$)

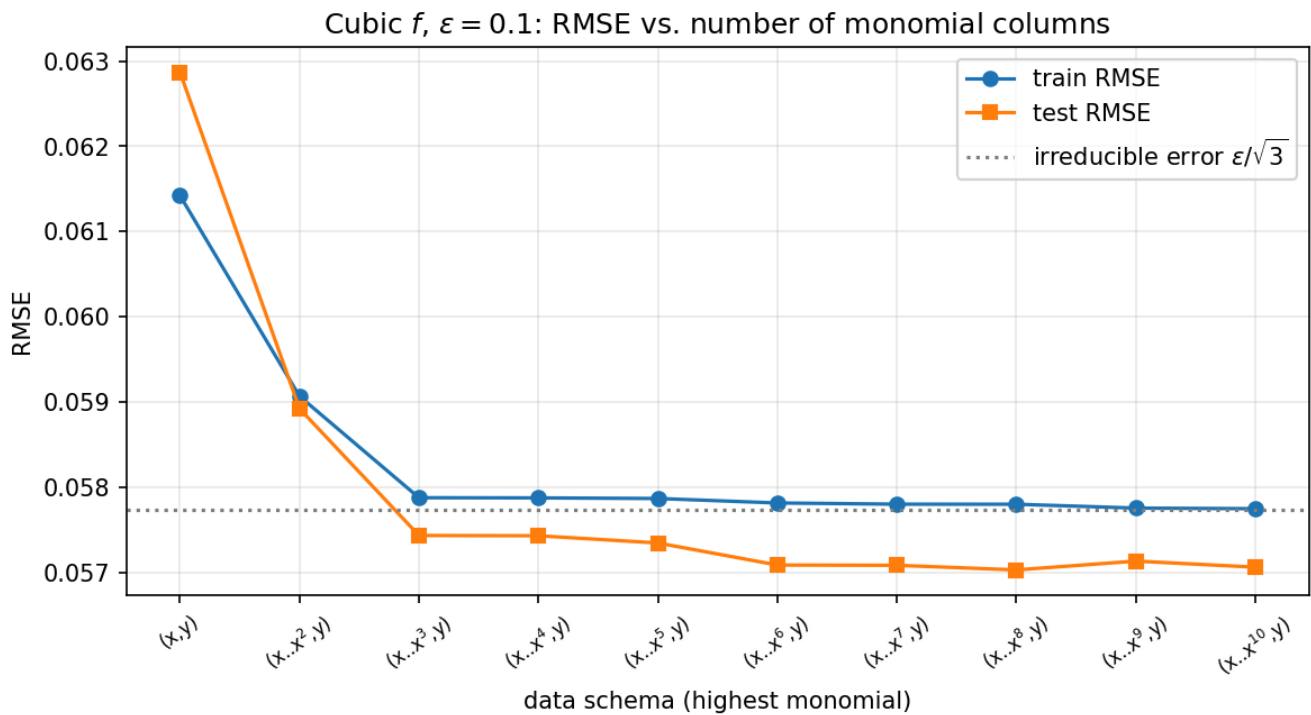


Figure 3. Train and test RMSE as columns $x^2, \dots, x^{10}$ are added; the dotted line is the irreducible error $0.1/\sqrt{3}$.

Columns	x	$x \dots x^2$	$x \dots x^3$	$x \dots x^4$	$x \dots x^5$	$x \dots x^6$	$x \dots x^7$	$x \dots x^8$	$x \dots x^9$	$x \dots x^{10}$
Train	0.06142	0.05907	0.05788	0.05788	0.05787	0.05782	0.05781	0.05781	0.05776	0.05775
Test	0.06287	0.05893	0.05744	0.05744	0.05735	0.05709	0.05709	0.05704	0.05714	0.05707

Training error never increases as columns are added. Each schema contains the previous one, so least squares can always reproduce the previous fit and can only fit the training data better.

Test error behaves differently:

- It drops sharply up to x^3 , because the bias disappears once the true cubic can be represented.
- After that it barely changes (≈ 0.0570 – 0.0574). The extra columns mostly fit noise, so the small additional training-error reductions do not carry over to the test data.

With 800 training points the variance added by a few extra coefficients is small, so clear overfitting (test error rising sharply) is not visible yet. With fewer records or higher degrees the gap between train and test error would grow. $X^T X$ also becomes badly conditioned ($\text{cond} \approx 5 \cdot 10^{14}$ at degree 10), which makes the inverse numerically fragile.

Q8. Pseudo-code: regression tree

```
class NODE: n_records, value (= mean y), feature, threshold, left, right    # leaf  $\Leftrightarrow$  left = None

function FIT(X, y, min_records):
    root  $\leftarrow$  BUILD(X, y)

function BUILD(X, y):
    node  $\leftarrow$  NODE(n_records = |y|, value = mean(y))
    if |y| < min_records or |y| < 2: return node                # stopping rule  $\rightarrow$  leaf
    split  $\leftarrow$  BEST_SPLIT(X, y)
    if split = None: return node                                # all y equal / all x equal  $\rightarrow$  leaf
    (j, t)  $\leftarrow$  split
    L  $\leftarrow$  {i : X[i, j]  $\leq$  t};    R  $\leftarrow$  {i : X[i, j] > t}
    node.feature, node.threshold  $\leftarrow$  j, t
    node.left  $\leftarrow$  BUILD(X[L], y[L])                          # recurse
    node.right  $\leftarrow$  BUILD(X[R], y[R])
    return node

function BEST_SPLIT(X, y):
    best  $\leftarrow$  ( $\infty$ , None)
    for each column j:
        sort records by X[:, j]  $\rightarrow$  x_s, y_s
        prefix sums S1[k] =  $\sum_{i \leq k} y_s[i]$ ,    S2[k] =  $\sum_{i \leq k} y_s[i]^2$ 
        for k = 1 .. n-1 with x_s[k] < x_s[k+1]:                # candidate cut between distinct
values
            SSE_L = S2[k] - S1[k]^2/k
            SSE_R = (S2[n]-S2[k]) - (S1[n]-S1[k])^2/(n-k)
            if SSE_L + SSE_R < best.sse:
                best  $\leftarrow$  (SSE_L + SSE_R, (j, (x_s[k] + x_s[k+1]) / 2))
    return best.split

function PREDICT(X):
    for each record x: node  $\leftarrow$  root
        while node is not a leaf: node  $\leftarrow$  node.left if x[node.feature]  $\leq$  node.threshold else
node.right
        output node.value

function VISUALIZE(ax, x_range, feature_funcs):
    grid  $\leftarrow$  dense x values in x_range;    G  $\leftarrow$  [grid, f_1(grid), ...]
    draw step plot of PREDICT(G) against grid
    for each internal node (j, t): x_split  $\leftarrow$  t if j = 0 else the x with f_j(x) = t    # monotone on
[0,1]
        draw vertical dashed line at x_split
```

Q9. Regression trees: all setup combinations

The trees use the same 800/200 splits as the linear models. In each figure the rows are the schemas and the columns are min_records = 1000, 50, 1. Each panel shows the training data, the true function (black), the tree prediction (red step line), and the split points (dashed gray). At min_records = 1 there are 799 split lines, so they form a gray band.

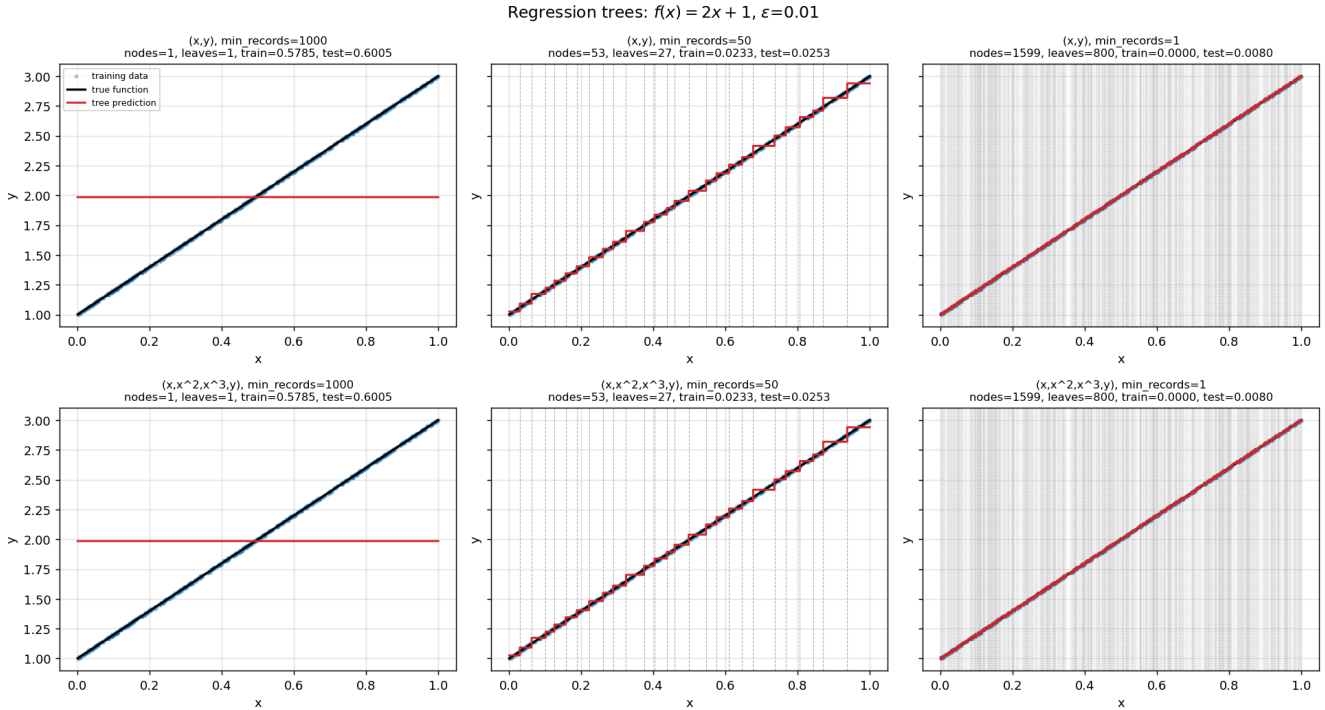


Figure 4. Linear f , $\varepsilon = 0.01$.

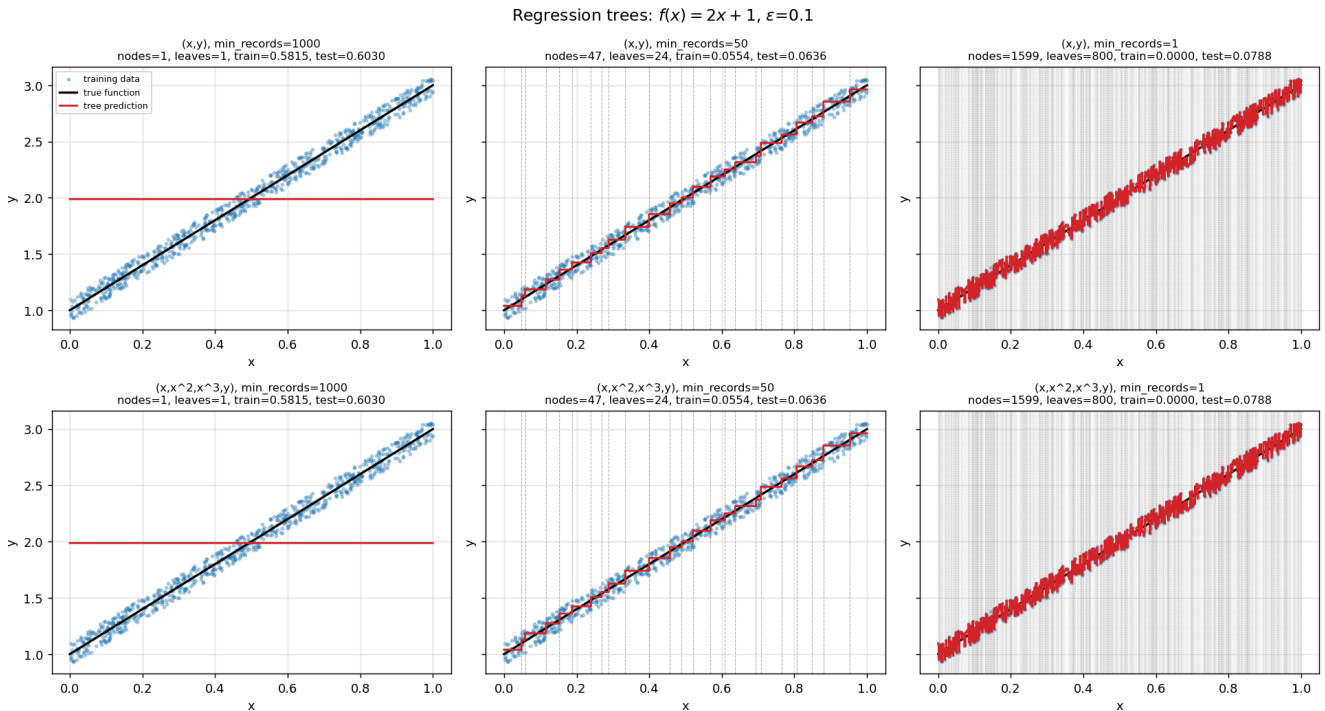


Figure 5. Linear f , $\varepsilon = 0.1$.

Regression trees: $f(x) = 0.5x^3 - x^2 + x$, $\varepsilon=0.01$

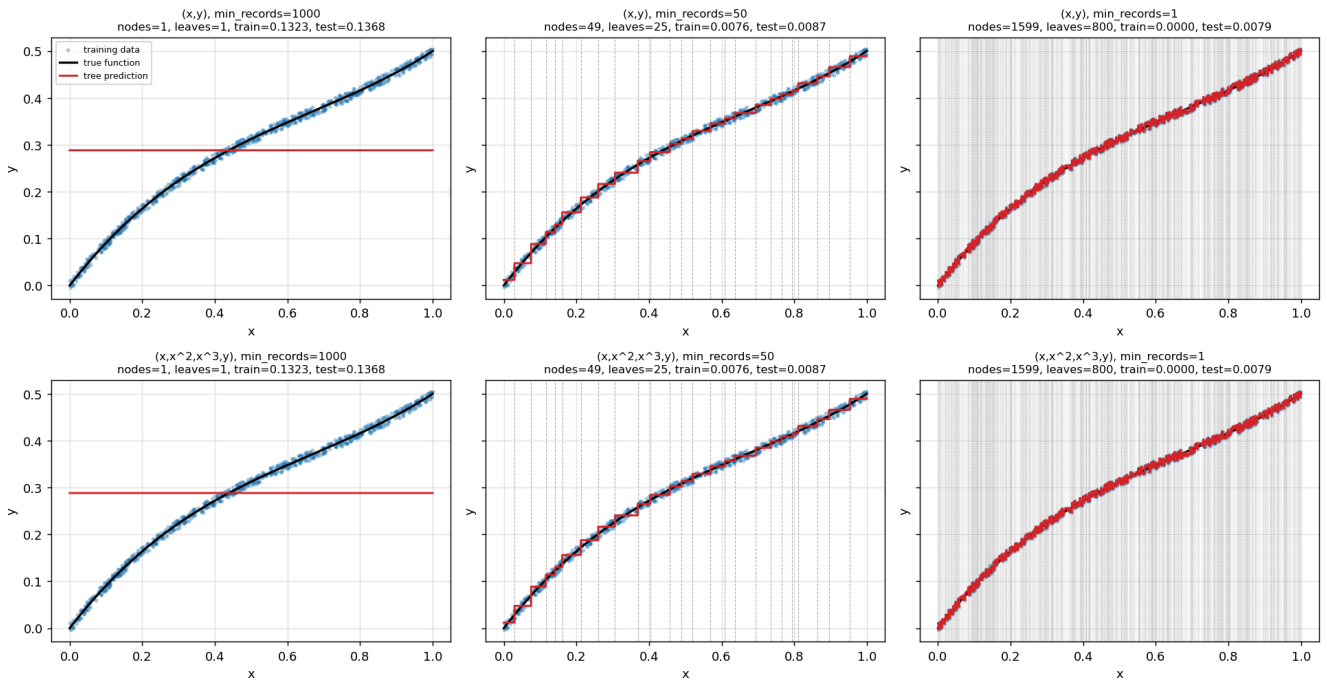


Figure 6. Cubic f , $\varepsilon = 0.01$.

Regression trees: $f(x) = 0.5x^3 - x^2 + x$, $\varepsilon=0.1$

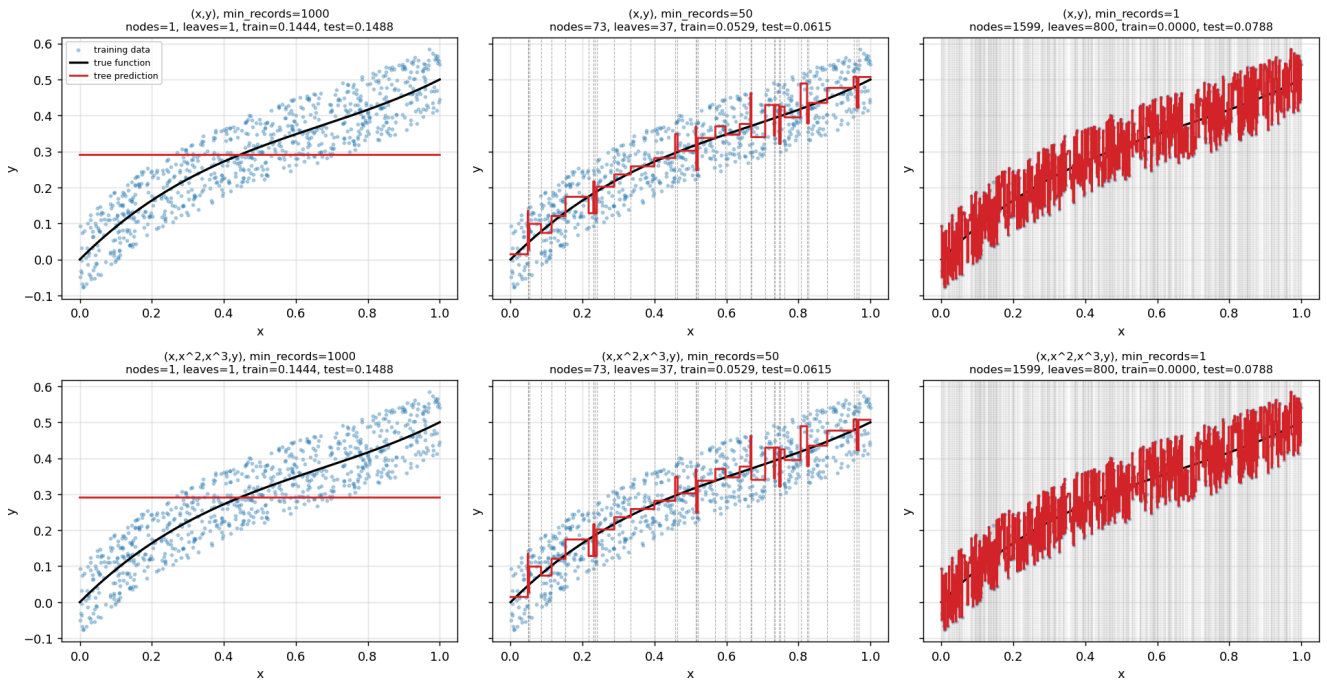


Figure 7. Cubic f , $\varepsilon = 0.1$.

Tree statistics and errors

True f	ε	Schema	min_records	Nodes	Leaves	Depth	Records per leaf	Train RMSE	Test RMSE
Linear $2x+1$	0.01	(x, y)	1000	1	1	0	800–800 (mean 800.0)	0.57854	0.60050
Linear $2x+1$	0.01	(x, y)	50	53	27	5	19–47 (mean 29.6)	0.02332	0.02530
Linear $2x+1$	0.01	(x, y)	1	1599	800	15	1–1 (mean 1.0)	0.00000	0.00803
Linear $2x+1$	0.01	(x, x^2, x^3, y)	1000	1	1	0	800–800 (mean 800.0)	0.57854	0.60050

Linear $2x+1$	0.01	(x, x^2, x^3, y)	50	53	27	5	19–47 (mean 29.6)	0.02332	0.02530
Linear $2x+1$	0.01	(x, x^2, x^3, y)	1	1599	800	15	1–1 (mean 1.0)	0.00000	0.00803
Linear $2x+1$	0.1	(x, y)	1000	1	1	0	800–800 (mean 800.0)	0.58155	0.60300
Linear $2x+1$	0.1	(x, y)	50	47	24	5	12–49 (mean 33.3)	0.05543	0.06361
Linear $2x+1$	0.1	(x, y)	1	1599	800	17	1–1 (mean 1.0)	0.00000	0.07879
Linear $2x+1$	0.1	(x, x^2, x^3, y)	1000	1	1	0	800–800 (mean 800.0)	0.58155	0.60300
Linear $2x+1$	0.1	(x, x^2, x^3, y)	50	47	24	5	12–49 (mean 33.3)	0.05543	0.06361
Linear $2x+1$	0.1	(x, x^2, x^3, y)	1	1599	800	17	1–1 (mean 1.0)	0.00000	0.07879
Cubic $0.5x^3-x^2+x$	0.01	(x, y)	1000	1	1	0	800–800 (mean 800.0)	0.13234	0.13685
Cubic $0.5x^3-x^2+x$	0.01	(x, y)	50	49	25	6	15–45 (mean 32.0)	0.00759	0.00873
Cubic $0.5x^3-x^2+x$	0.01	(x, y)	1	1599	800	16	1–1 (mean 1.0)	0.00000	0.00788
Cubic $0.5x^3-x^2+x$	0.01	(x, x^2, x^3, y)	1000	1	1	0	800–800 (mean 800.0)	0.13234	0.13685
Cubic $0.5x^3-x^2+x$	0.01	(x, x^2, x^3, y)	50	49	25	6	15–45 (mean 32.0)	0.00759	0.00873
Cubic $0.5x^3-x^2+x$	0.01	(x, x^2, x^3, y)	1	1599	800	16	1–1 (mean 1.0)	0.00000	0.00788
Cubic $0.5x^3-x^2+x$	0.1	(x, y)	1000	1	1	0	800–800 (mean 800.0)	0.14444	0.14877
Cubic $0.5x^3-x^2+x$	0.1	(x, y)	50	73	37	9	1–49 (mean 21.6)	0.05294	0.06155
Cubic $0.5x^3-x^2+x$	0.1	(x, y)	1	1599	800	22	1–1 (mean 1.0)	0.00000	0.07883
Cubic $0.5x^3-x^2+x$	0.1	(x, x^2, x^3, y)	1000	1	1	0	800–800 (mean 800.0)	0.14444	0.14877
Cubic $0.5x^3-x^2+x$	0.1	(x, x^2, x^3, y)	50	73	37	9	1–49 (mean 21.6)	0.05294	0.06155
Cubic $0.5x^3-x^2+x$	0.1	(x, x^2, x^3, y)	1	1599	800	22	1–1 (mean 1.0)	0.00000	0.07883

Q10. Trees vs. linear models (test RMSE)

True f	ϵ	Schema	Linear	Tree m=1000	Tree m=50	Tree m=1	Winner
Linear $2x+1$	0.01	(x, y)	0.00574	0.60050	0.02530	0.00803	linear
Linear $2x+1$	0.01	(x, x^2, x^3, y)	0.00574	0.60050	0.02530	0.00803	linear
Linear $2x+1$	0.1	(x, y)	0.05739	0.60300	0.06361	0.07879	linear
Linear $2x+1$	0.1	(x, x^2, x^3, y)	0.05744	0.60300	0.06361	0.07879	linear
Cubic $0.5x^3 - x^2 + x$	0.01	(x, y)	0.02214	0.13685	0.00873	0.00788	tree (m=1)
Cubic $0.5x^3 - x^2 + x$	0.01	(x, x^2, x^3, y)	0.00574	0.13685	0.00873	0.00788	linear
Cubic $0.5x^3 - x^2 + x$	0.1	(x, y)	0.06287	0.14877	0.06155	0.07883	tree (m=50)
Cubic $0.5x^3 - x^2 + x$	0.1	(x, x^2, x^3, y)	0.05744	0.14877	0.06155	0.07883	linear

Who wins? The **linear model wins whenever its schema contains the true function**: the linear truth with both schemas and the cubic truth with (x, x^2, x^3) . It then sits at the noise floor, which a piecewise-constant tree cannot reach with 800 records. The **tree wins only for the cubic truth on (x, y)** , where the straight line is biased: 0.0079 vs 0.0221 at $\epsilon = 0.01$, and 0.062 vs 0.063 at $\epsilon = 0.1$.

(i) Extra columns. They make **no difference** to the trees: every tree, RMSE, node count, and leaf is identical for the two schemas.

- On $[0, 1]$, x^2 and x^3 are strictly increasing functions of x .
- So a threshold on x^2 or x^3 produces exactly the same left/right partition as some threshold on x .
- Trees only use the ordering of a column, so they gain nothing; our implementation always chose column x .

Linear regression, by contrast, benefits from the extra columns because it uses their actual values.

(ii) Tree size. Size matters a lot:

- **min_records = 1000:** the 800-record root is never split, giving **1 node** (a single leaf with 800 records). The prediction is a constant (the mean of y), with high bias: test RMSE 0.60 (linear) and 0.14 (cubic).
- **min_records = 50: 47–73 nodes** (24–37 leaves, depth 5–9). Leaves hold mostly 12–49 records, occasionally fewer, because a split of a ≥ 50 -record node can produce a very small child. This gives a good staircase approximation.
- **min_records = 1:** splitting continues until every leaf is pure, giving **1599 nodes** (800 leaves of exactly 1 record each, depth 15–22). Training RMSE is 0 because the tree memorizes the data, noise included.

The best size is in between and depends on the noise.

(iii) Noise level. The noise level decides which tree size is best:

- **$\epsilon = 0.01$:** memorizing the noise is cheap, and the main error comes from the step approximation. The fully grown tree ($m = 1$) is therefore the best tree (0.0080 linear, 0.0079 cubic), clearly better than $m = 50$ (0.025, 0.0087).
- **$\epsilon = 0.1$:** the noise dominates. A 1-record leaf passes its noise directly into predictions, giving test RMSE $0.079 \approx \sqrt{2 \cdot \epsilon / 3}$. The $m = 50$ tree averages the noise over many records per leaf and wins (0.064, 0.062).

Noise also raises every model's error floor ($\times 10$ from $\epsilon = 0.01$ to 0.1), which narrows the gap between the trees and the biased linear model.

Q11. Overfitting study: $f(x) = \sin(2\pi x)$, $n = 200$, $\varepsilon = 0.2$

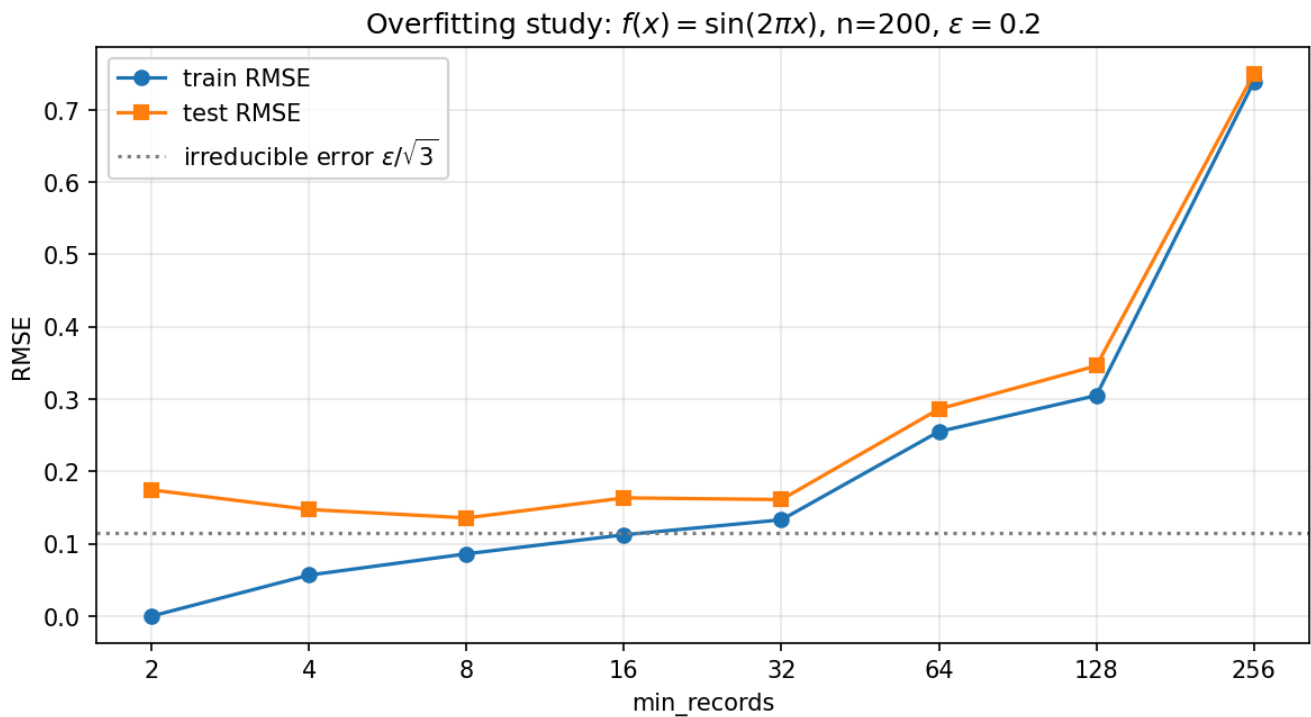


Figure 8. Train and test RMSE vs. min_records (160 training / 40 test records). Dotted: irreducible error $0.2/\sqrt{3} \approx 0.115$.

min_records	Leaves	Train RMSE	Test RMSE
2	160	0.0000	0.1746
4	74	0.0567	0.1475
8	37	0.0862	0.1358
16	17	0.1125	0.1635
32	10	0.1332	0.1611
64	4	0.2552	0.2863
128	2	0.3050	0.3461
256	1	0.7389	0.7497

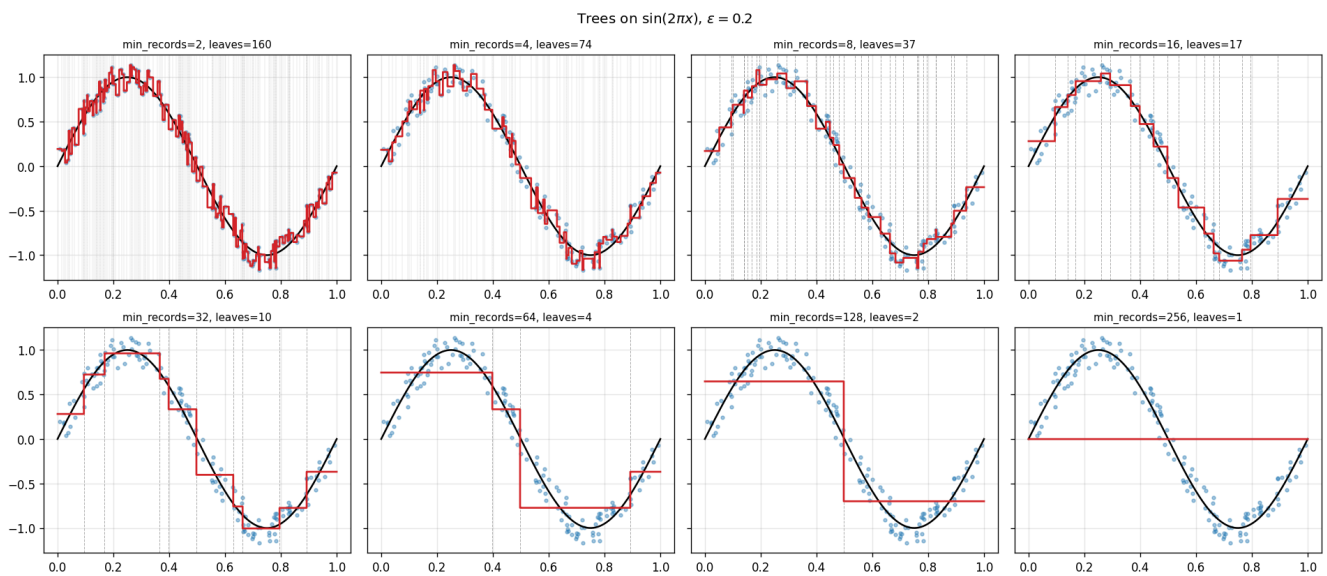


Figure 9. The fitted trees for each min_records value.

- **Best model: min_records = 8**, with the lowest test RMSE (0.136, 37 leaves). It follows the sine wave without chasing individual noisy points. Values 16–32 are close behind (0.161–0.164); with only 40 test records, these differences are partly noise.
- **Highest bias: min_records = 256**. With only 160 training records the root is never split, so the tree is the constant mean of y . It cannot represent the sine shape at all, and both errors are high (train 0.739, test 0.750, nearly equal). min_records = 128 (2 leaves) is also strongly biased.
- **Highest variance: min_records = 2**. The tree grows until every leaf holds one record (160 leaves), so it fits the noise exactly: training RMSE is 0 but test RMSE is 0.175. A different noise sample would produce a completely different tree.

In between, training error rises steadily with min_records, while test error first falls and then rises: the classic U-shaped bias-variance trade-off.

Q12. Hyper-parameters

Linear regression: the training algorithm itself has **no tunable hyper-parameters**. β comes from the closed-form normal equation, so there is no learning rate, number of iterations, or regularization strength. The only model-complexity knob is the **choice of input columns** (the feature_funcs, e.g. the highest monomial degree in Q7). It is a modelling or feature-engineering choice that behaves like a hyper-parameter: it controls the bias-variance trade-off.

Regression tree: one hyper-parameter, **min_records**. A node is split only if it holds at least that many training records, so it controls tree size and hence bias vs. variance. Everything else is fixed by design: SSE split criterion, thresholds at midpoints, no maximum depth, and no pruning.

Not hyper-parameters: the data-generation settings (range, sample count, noise ϵ), the 80/20 split fraction, and the random seeds are experiment settings, not hyper-parameters of either algorithm.